Sentiment Analysis on Movie Reviews using Bidirectional LSTM

Objective

To build and evaluate a BiLSTM model for binary sentiment classification on movie reviews.

Dataset Used

The IMDB Movie Reviews dataset. This dataset contains 50,000 highly polarized movie reviews, split equally into 25,000 reviews for training and 25,000 for testing. Each review is labelled as either positive or negative.

- **Source**: Kaggle Dataset – IMDB Large Movie Reviews Sentiment Dataset
- **Description**: Contains 50,000 highly polar movie reviews, evenly split into 25,000 positive and 25,000 negative samples.
  – **Training Set:** 25,000 reviews (balanced positive/negative).
  – **Test Set**: 25,000 reviews (balanced positive/negative).
- **Features**: Raw text reviews of variable length (most exceed 200 words).
- **Classes**: 2 (positive, negative).

Theory

Sentiment analysis is the task of determining the emotional tone or sentiment expressed in a piece of text. Recurrent Neural Networks (RNNs) are well-suited for sequential data like text, as they can capture dependencies between words. However, standard RNNs can struggle with capturing long-term dependencies due to the vanishing gradient problem. Long Short-Term Memory (LSTM) networks are a type of RNN specifically designed to address this issue through the use of gating mechanisms (input, forget, and output gates) that regulate the flow of information and maintain a memory cell.

A standard LSTM processes a sequence in only one direction (e.g., from beginning to end). In sentiment analysis, the context of a word can be influenced by words that come both before and after it. A Bidirectional LSTM (BiLSTM) enhances the standard LSTM by processing the sequence in both forward and backward directions independently and then concatenating or combining their outputs. This allows the model to capture context from both past and future words in the sequence, leading to a richer understanding of the text and potentially improved performance in sentiment analysis tasks. The outputs from the forward and backward LSTMs at each time step are typically combined before being fed into subsequent layers for classification.

Code

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from datasets import load_dataset
import matplotlib.pyplot as plt
import numpy as np
import nltk
from nltk.tokenize import word_tokenize
```

```
from collections import Counter
import uuid
# Download NLTK tokenizer data
nltk.download('punkt')
# Set random seed for reproducibility
torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Hyperparameters
MAX_VOCAB_SIZE = 25000
MAX_SEQUENCE_LENGTH = 500
EMBEDDING_DIM = 100
HIDDEN_DIM = 256
OUTPUT_DIM = 1
N_LAYERS = 2
DROPOUT = 0.5
BATCH_SIZE = 64
N_EPOCHS = 5
# Load IMDB dataset using datasets
dataset = load_dataset("imdb")
train_data = dataset["train"]
test_data = dataset["test"]
# Build vocabulary
def build_vocab(texts, max_size):
    counter = Counter()
    for text in texts:
        tokens = word_tokenize(text.lower())
        counter.update(tokens)
    # Select most common tokens
    most_common = counter.most_common(max_size - 2)# Reserve2for<unk>and<pad>
    vocab = {"<unk>": 0, "<pad>": 1}
    for i, (word, _) in enumerate(most_common, 2):
        vocab[word] = i
    return vocab
# Create vocabulary from training data
vocab = build_vocab([item["text"] for item in train_data], MAX_VOCAB_SIZE)
# Text and label processing
def text_pipeline(text, vocab):
    tokens = word_tokenize(text.lower())
    indices = [vocab.get(token, vocab["<unk>"]) for token in
tokens][:MAX_SEQUENCE_LENGTH]
    if len(indices) < MAX_SEQUENCE_LENGTH:
        indices += [vocab["<pad>"]] * (MAX_SEQUENCE_LENGTH - len(indices))
    return indices
def label_pipeline(label):
    return float(label)
# Custom Dataset class
class IMDBDataset(Dataset):
    def __init__(self, data, vocab):
```

```
        self.data = data
        self.vocab = vocab
    def __len__(self):
        return len(self.data)
    def __getitem__(self, idx):
        text = self.data[idx]["text"]
        label = self.data[idx]["label"]
        return text_pipeline(text, self.vocab), label_pipeline(label)
# Create data loaders
train_dataset = IMDBDataset(train_data, vocab)
test_dataset = IMDBDataset(test_data, vocab)
def collate_batch(batch):
    text_list, label_list = zip(*batch)
    text_tensor = torch.tensor(text_list, dtype=torch.long).to(device)
    label_tensor = torch.tensor(label_list, dtype=torch.float32).to(device)
    return text_tensor, label_tenso
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
collate_fn=collate_batch)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
collate_fn=collate_batch)
# Define BiLSTM model
class BiLSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
n_layers, dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=vocab["<pad>"])
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            num_layers=n_layers,
            bidirectional=True,
            dropout=dropout if n_layers > 1 else 0,
            batch_first=True)
        self.fc = nn.Linear(hidden_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)
        self.sigmoid = nn.Sigmoid()
    def forward(self, text):
        embedded = self.dropout(self.embedding(text))
        output, (hidden, cell) = self.lstm(embedded)
        hidden = self.dropout(torch.cat((hidden[-2, :, :], hidden[-1, :, :]),
dim=1))
        dense = self.fc(hidden)
        return self.sigmoid(dense)
# Initialize model
model = BiLSTM(
    vocab_size=len(vocab),
    embedding_dim=EMBEDDING_DIM,
```

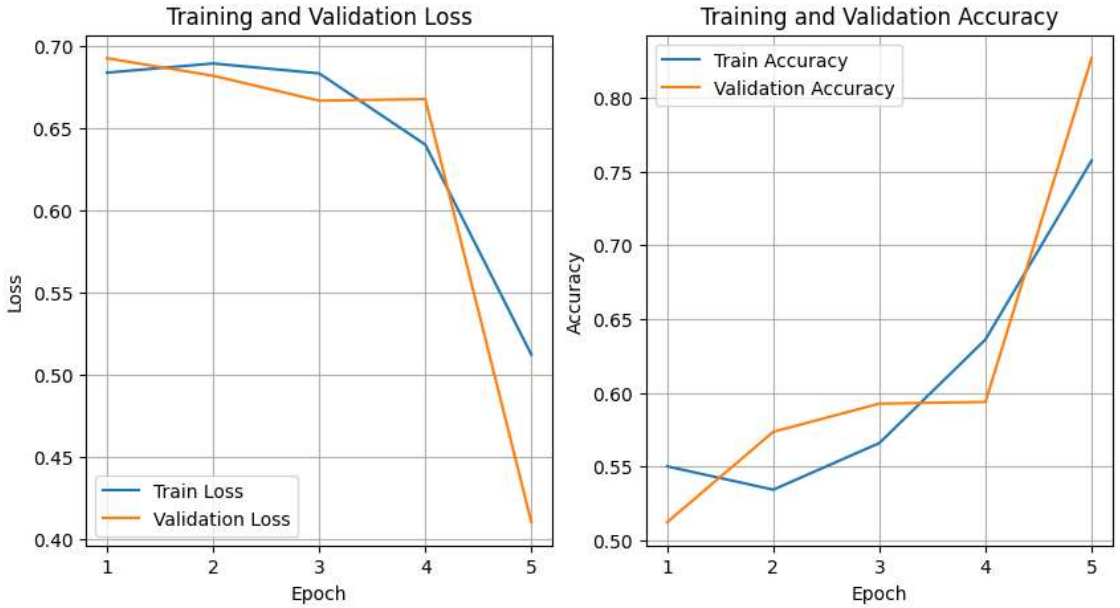
```
    hidden_dim=HIDDEN_DIM,
    output_dim=OUTPUT_DIM,
    n_layers=N_LAYERS,
    dropout=DROPOUT).to(device)
# Loss and optimizer
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters())
# Training function
def train(model, iterator, optimizer, criterion):
    model.train()
    epoch_loss = 0
    epoch_acc = 0
    total = 0
    for text, labels in iterator:
        optimizer.zero_grad()
        predictions = model(text).squeeze(1)
        loss = criterion(predictions, labels)
        acc = ((predictions > 0.5).float() == labels).float().mean()
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(labels)
        epoch_acc += acc.item() * len(labels)
        total += len(labels)
    return epoch_loss / total, epoch_acc / total
# Evaluation function
def evaluate(model, iterator, criterion):
    model.eval()
    epoch_loss = 0
    epoch_acc = 0
    total = 0
    with torch.no_grad():
        for text, labels in iterator:
            predictions = model(text).squeeze(1)
            loss = criterion(predictions, labels)
            acc = ((predictions > 0.5).float() == labels).float().mean()
            epoch_loss += loss.item() * len(labels)
            epoch_acc += acc.item() * len(labels)
            total += len(labels)

    return epoch_loss / total, epoch_acc / total
# Training loop
train_losses, train_accs = [], []
val_losses, val_accs = [], []
for epoch in range(N_EPOCHS):
    train_loss, train_acc = train(model, train_loader, optimizer, criterion)
    val_loss, val_acc = evaluate(model, test_loader, criterion)
    train_losses.append(train_loss)
    train_accs.append(train_acc)
```

```
    val_losses.append(val_loss)
    val_accs.append(val_acc)
    print(f'Epoch: {epoch+1:02}')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
    print(f'\tVal Loss: {val_loss:.3f} | Val Acc: {val_acc*100:.2f}%')
plt.figure(figsize=(10, 5))
# Loss plot
plt.subplot(1, 2, 1)
plt.plot(range(1, N_EPOCHS+1), train_losses, label='Train Loss')
plt.plot(range(1, N_EPOCHS+1), val_losses, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.savefig('loss_plot.png')
# Accuracy plot
plt.subplot(1, 2, 2)
plt.plot(range(1, N_EPOCHS+1), train_accs, label='Train Accuracy')
plt.plot(range(1, N_EPOCHS+1), val_accs, label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.savefig('accuracy_plot.png')
```

Output

```
[nltk_data] Downloading package punkt to /usr/share/nltk_data...
[nltk_data]   Package punkt is already up-to-date!

README.md: 100% 7.81k/7.81k [00:00<00:00, 814kB/s]

train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 208MB/s]

test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 130MB/s]

unsupervised-00000-
of-00001.parquet: 100% 42.0M/42.0M [00:00<00:00, 347MB/
s]

Generating train split: 100% 25000/25000 [00:00<00:00, 89441.28 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 177411.27 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 202300.33 examples/
```

```
Epoch: 01
        Train Loss: 0.684 | Train Acc: 55.00%
        Val Loss: 0.693 | Val Acc: 51.22%
Epoch: 02
        Train Loss: 0.689 | Train Acc: 53.42%
        Val Loss: 0.682 | Val Acc: 57.35%
Epoch: 03
        Train Loss: 0.683 | Train Acc: 56.58%
        Val Loss: 0.667 | Val Acc: 59.26%
Epoch: 04
        Train Loss: 0.640 | Train Acc: 63.61%
        Val Loss: 0.668 | Val Acc: 59.38%
Epoch: 05
        Train Loss: 0.512 | Train Acc: 75.74%
        Val Loss: 0.410 | Val Acc: 82.68%
```



Conclusion

The experiment will demonstrate the effectiveness of using a Bidirectional LSTM network for sentiment analysis on movie reviews. A well-trained BiLSTM model is expected to achieve a reasonably high accuracy in classifying the sentiment of unseen movie reviews by leveraging contextual information from both directions of the text sequence. The performance metrics obtained will indicate the model's ability to generalize to new data.